Case 02 · Ideation document: Private Exposure Registry

Following chapter 1 (Initiation) of the Magshimim project-book template. For each project: overview, goal, target audience, existing solutions, technological core, technologies, challenges and sources, hardware requirements, and conclusion.

Overview

A system that lets financial institutions combine their exposure data into a shared picture without revealing it, using secure multi-party computation (MPC), so that risk building up across the whole market can be seen while each participant sees only its own data.

In practice: each participant (a credit fund, a bank, a pension fund) "splits" its data into random pieces and sends each piece to a different server. No single server can reconstruct anything. The servers jointly compute, on the split pieces, questions such as "how much leveraged credit in the market depends on rates above 5%?" or "is there one sector every fund is exposed to?". Only the aggregate answer is opened.

Project goal

We chose this project because the story points to a paradox: the greatest danger is the exposure nobody saw, but the reason nobody saw it is that the data are secret. A fund will not reveal its positions to competitors, and sometimes not to a foreign regulator either. The result is a market (private credit) in which everyone knows everything about themselves and nobody knows anything about the whole.

The goal: to show that this paradox can be dissolved. There is no need to choose between secrecy and transparency: one can compute on data without seeing them.

Target audience

- **Primary audience:** supervisors and central banks (in Israel, Europe and America) who need an aggregate picture of a market whose data are spread across institutions and countries. Required knowledge: an understanding of financial risk; the cryptography is invisible to them.
- **Secondary audience:** industry associations and groups of institutions who want to compare themselves with the sector average ("are we more exposed than the others?") without revealing data.
- **Illustrative scenario:** three regulators (Britain, France, America) want to know whether the same leveraged credit funds are exposed to the same companies in all three countries. None of them is allowed to send its data to the others. They run the question through the registry and receive only the joint answer.

Existing solutions

- **Form PF (America):** large private funds report exposures and leverage to the SEC in confidence. This shows that regulators already collect the information, but each in its own silo, and sharing between regulators is legally limited.
- Abbe, Khandani and Lo, "Privacy-Preserving Methods for Sharing Financial Risk Exposures" (2012): a paper proposing exactly this idea, MPC for computing systemic risk. An academic proof of feasibility, with no deployment in the field.
- **Boston Women's Workforce Council (2015 onwards):** Boston employers shared pay data through MPC to measure pay gaps, without any employer revealing its data. Proof that competing organisations will take part when there is a trusted convener and a shared goal.
- **Sharemind:** a commercial MPC platform used in Estonia to analyse tax and education data. Proof that the technology works at government scale.
- **Open MPC libraries:** MP-SPDZ, MPyC. Building blocks, not a solution.

What sets us apart: a focused application of MPC to the specific problem of rate and leveraged-credit exposures, with a predefined set of questions a regulator actually needs, and a layer that protects the output (so that the answer itself does not give a participant away).

Technological core

The core is the joint-computation protocol: splitting each value into random shares (secret sharing) so that each server holds a share that looks like noise, and performing additions, comparisons and multiplications directly on the shares. Only at the end, when all servers combine their shares of the result, is a single number opened: the aggregate answer.

Without this protocol there is no project: everything else (the query interface, the reports, output protection) is built on it.

Main technologies

- **MPyC:** a Python MPC library based on Shamir secret sharing. We chose it because it is readable, documented, and suits a POC in which we want to show every step.
- **Differential privacy (via OpenDP):** adding controlled noise to the result so that a single participant's data cannot be extracted from the aggregate (for example, if a sector has only three funds).
- **gRPC with mutual TLS:** for communication between compute servers, with every party authenticated.
- **Python and FastAPI:** for the interface where a participant uploads data and a regulator asks a question.

Open questions:

- **MPC versus homomorphic encryption (OpenFHE, TenSEAL):** homomorphic encryption lets a single server compute on encrypted data, with no need for several non-colluding servers. But it is far slower and limited in the kinds of computation it supports. MPC is faster but requires the assumption that servers do not collude.

- **MPC versus hardware enclaves (Intel SGX, AMD SEV):** very fast, but requires trusting the chip maker, and known vulnerabilities have been found in them.

Technological challenges and sources

- **The incentive problem:** why would a fund take part? This is not a technical challenge but a structural one, and it must be answered right here. Our answer: the participants are not volunteer funds but regulators that already collect the data by law (as with Form PF). The system does not ask for new information; it allows information that is already collected, but cannot move between authorities, to be combined. The Boston example shows that voluntary participation is also possible with a trusted convener.
- **Performance:** comparisons (for example "is the rate above 5%?") are far more expensive in MPC than additions. We must choose questions that can be computed in reasonable time, and measure.
- **The non-collusion assumption:** security rests on the servers not combining their shares. We must define who runs each server (for example, each regulator runs one).
- **Leakage through the result:** even an aggregate answer can give a participant away when the group is small. Hence differential privacy and a minimum number of participants per question.

Sources: MPyC documentation (github.com/lschoe/mpyc), MP-SPDZ (github.com/data61/MP-SPDZ), the paper by Abbe, Khandani and Lo, and the OpenDP documentation.

Hardware requirements

No special hardware. For the POC, three processes on one machine simulating three servers are enough. In a real deployment, each server sits with a different organisation.

Conclusion

✅ Survives, with a condition. The cryptographic foundation is proven, and there are real precedents for its use. The condition: the incentive problem. This document proposes an answer (regulators that already collect by law), and in the research stage we must check that the answer holds and that the computation is fast enough for a regulator to use.